

小结

131

`string` 和 `vector` 是两种最重要的标准库类型。`string` 对象是一个可变长的字符序列，`vector` 对象是一组同类型对象的容器。

迭代器允许对容器中的对象进行间接访问，对于 `string` 对象和 `vector` 对象来说，可以通过迭代器访问元素或者在元素间移动。

数组和指向数组元素的指针在一个较低的层次上实现了与标准库类型 `string` 和 `vector` 类似的功能。一般来说，应该优先选用标准库提供的类型，之后再考虑 C++ 语言内置的低层的替代品数组或指针。

术语表

begin 是 `string` 和 `vector` 的成员，返回指向第一个元素的迭代器。也是一个标准库函数，输入一个数组，返回指向该数组首元素的指针。

缓冲区溢出 (buffer overflow) 一种严重的程序故障，主要的原因是试图通过一个越界的索引访问容器内容，容器类型包括 `string`、`vector` 和数组等。

C 风格字符串 (C-style string) 以空字符结束的字符数组。字符串字面值是 C 风格字符串，C 风格字符串容易出错。

类模板 (class template) 用于创建具体类类型的模板。要想使用类模板，必须提供关于类型的辅助信息。例如，要定义一个 `vector` 对象需要指定元素的类型：`vector<int>` 包含 `int` 类型的元素。

编译器扩展 (compiler extension) 某个特定的编译器为 C++ 语言额外增加的特性。基于编译器扩展编写的程序不易移植到其他编译器上。

容器 (container) 是一种类型，其对象容纳了一组给定类型的对象。`vector` 是一种容器类型。

拷贝初始化 (copy initialization) 使用赋值号 (=) 的初始化形式。新创建的对象是初始值的一个副本。

difference_type 由 `string` 和 `vector` 定义的一种带符号整数类型，表示两个迭代

器之间的距离。

直接初始化 (direct initialization) 不使用赋值号 (=) 的初始化形式。

empty 是 `string` 和 `vector` 的成员，返回一个布尔值。当对象的大小为 0 时返回真，否则返回假。

end 是 `string` 和 `vector` 的成员，返回一个尾后迭代器。也是一个标准库函数，输入一个数组，返回指向该数组尾元素的下一位置的指针。

getline 在 `string` 头文件中定义的一个函数，以一个 `istream` 对象和一个 `string` 对象为输入参数。该函数首先读取输入流的内容直到遇到换行符停止，然后将读入的数据存入 `string` 对象，最后返回 `istream` 对象。其中换行符读入但是不保留。

索引 (index) 是下标运算符使用的值。表示要在 `string` 对象、`vector` 对象或者数组中访问的一个位置。

实例化 (instantiation) 编译器生成一个指定的模板类或函数的过程。

迭代器 (iterator) 是一种类型，用于访问容器中的元素或者在元素之间移动。

迭代器运算 (iterator arithmetic) 是 `string` 或 `vector` 的迭代器的运算：迭代器与整数相加或相减得到一个新的迭代器，与原来的迭代器相比，新迭代器向前

或向后移动了若干个位置。两个迭代器相减得到它们之间的距离，此时它们必须指向同一个容器的元素或该容器尾元素的下一位置。

132 以空字符结束的字符串（null-terminated string）是一个字符串，它的最后一个字符后面还跟着一个空字符（'\0'）。

尾迭代器（off-the-end iterator） end 函数返回的迭代器，指向一个并不存在的元素，该元素位于容器尾元素的下一位置。

指针运算（pointer arithmetic） 是指针类型支持的算术运算。指向数组的指针所支持的运算种类与迭代器运算一样。

ptrdiff_t 是 cstdint 头文件中定义的一种与机器实现有关的带符号整数类型，它的空间足够大，能够表示数组中任意两个指针之间的距离。

push_back 是 vector 的成员，向 vector 对象的末尾添加元素。

范围 for 语句（range for） 一种控制语句，可以在值的一个特定集合内迭代。

size 是 string 和 vector 的成员，分别返回字符的数量或元素的数量。返回值的类型是 size_type。

size_t 是 cstdint 头文件中定义的一种与机器实现有关的无符号整数类型，它的空间足够大，能够表示任意数组的大小。

size_type 是 string 和 vector 定义的类型名字，能存放下任意 string 对象或 vector 对象的大小。在标准库中，size_type 被定义为无符号类型。

string 是一种标准库类型，表示字符的序列。

using 声明（using declaration） 令命名空间中的某个名字可被程序直接使用。

```
using 命名空间 :: 名字;
```

上述语句的作用是令程序可以直接使用名字，而无须写它的前缀部分命名空间::。

值初始化（value initialization） 是一种初始化过程。内置类型初始化为 0，类类型由

类的默认构造函数初始化。只有当类包含默认构造函数时，该类的对象才会被值初始化。对于容器的初始化来说，如果只说明了容器的大小而没有指定初始值的话，就会执行值初始化。此时编译器会生成一个值，而容器的元素被初始化为该值。

vector 是一种标准库类型，容纳某指定类型的一组元素。

++运算符（++ operator） 是迭代器和指针定义的递增运算符。执行“加 1”操作使得迭代器指向下一个元素。

[]运算符（[] operator） 下标运算符。obj[j] 得到容器对象 obj 中位置 j 的那个元素。索引从 0 开始，第一个元素的索引是 0，尾元素的索引是 obj.size()-1。下标运算符的返回值是一个对象。如果 p 是指针，n 是整数，则 p[n] 与 *(p+n) 等价。

->运算符（-> operator） 箭头运算符，该运算符综合了解引用操作和点操作。a->b 等价于 (*a).b。

<<运算符（<< operator） 标准库类型 string 定义的输出运算符，负责输出 string 对象中的字符。

>>运算符（>> operator） 标准库类型 string 定义的输入运算符，负责读入一组字符，遇到空白停止，读入的内容赋给运算符右侧的运算对象，该运算对象应该是一个 string 对象。

!运算符（! operator） 逻辑非运算符，将它的运算对象的布尔值取反。如果运算对象是假，则结果为真，如果运算对象是真，则结果为假。

&&运算符（&& operator） 逻辑与运算符，如果两个运算对象都是真，结果为真。只有当左侧运算对象为真时才会检查右侧运算对象。

||运算符（|| operator） 逻辑或运算符，任何一个运算对象是真，结果就为真。只有当左侧运算对象为假时才会检查右侧运算对象。

第 4 章

表达式

内容

4.1 基础.....	120
4.2 算术运算符.....	124
4.3 逻辑和关系运算符.....	126
4.4 赋值运算符.....	129
4.5 递增和递减运算符.....	131
4.6 成员访问运算符.....	133
4.7 条件运算符.....	134
4.8 位运算符.....	135
4.9 sizeof 运算符.....	139
4.10 逗号运算符.....	140
4.11 类型转换.....	141
4.12 运算符优先级表.....	147
小结.....	149
术语表.....	149

C++语言提供了一套丰富的运算符，并定义了这些运算符作用于内置类型的运算对象时所执行的操作。同时，当运算对象是类类型时，C++语言也允许由用户指定上述运算符的含义。本章主要介绍由语言本身定义、并用于内置类型运算对象的运算符，同时简单介绍几种标准库定义的运算符。第 14 章会专门介绍用户如何自定义适用于类类型的运算符。